Case study: building a test framework teams can actually adopt

Why I built this

After many years in test automation I've seen the same thing at most companies: a suite that starts out fine, then slowly becomes something nobody trusts. Tests fail for no reason, test data runs out, the report is unreadable to anyone who isn't an engineer, and the build only runs on one person's machine.

I wanted a starting point that avoids those problems from day one, and I wanted to prove it on a real application, not a login-page demo. So I picked Toolshop, an open-source web shop with a customer front end, an admin area, a REST API and a database. It has enough real behaviour (carts, five payment methods, roles, stock, caching) to show where automation gets hard.

What I set out to do

- Test the application at both levels, the browser and the API, in one project and one report.
- Make every run repeatable, whether on my laptop, a teammate's or a CI server.
- Keep tests independent, so they can run in parallel without stepping on each other's data.
- Produce a report a project manager can read.
- Make it easy to swap in a different application later.

How it's put together

The code is split into layers that only depend downward: tests, then fixtures, then page objects and API clients, then configuration. The bottom layers know nothing about Toolshop, so they carry over to another product unchanged. ARCHITECTURE.md has the details.

A few choices worth explaining:

- **Orders are verified through the API, not only on screen.** After a UI checkout, the test asks the invoices API what was actually saved. A page that says "thank you" proves very little.
- **Tests create their own data and remove it.** Fixtures build brands, products and users through the API and delete them afterwards, even when the test fails. Nothing relies on data another test might change.
- **The application starts in CI from the repo itself.** Docker Compose, with the images pinned by digest, so a run never depends on anyone's machine and the application can't change under the tests.

- **Some things are checked in the database.** The API says a user was created, but only the database shows whether the password was stored hashed, or whether an order has its items and payment rows.
- **Real defects are recorded, not hidden.** When a test finds a bug in the application, it's written with the correct expectation and marked as an expected failure. The build stays green while the bug exists and the test fails loudly the day it's fixed.

Problems I ran into

Most of the useful lessons came from things going wrong:

- **Stock ran out.** Seeded products have limited stock, and an ordered product can't be deleted. After a few runs, order tests started failing. I added one shared product with a very large stock that a setup step creates when it's missing.
- **A cache that nobody invalidates.** Product search results are cached by the server and re-seeding the database doesn't clear them. After a reset, tests saw old data. The start script now clears the cache.
- **Parallel runs overloaded the environment.** Four workers against one dev server caused timeouts that looked like test failures. Worker count is now configurable.
- **A test that blamed the wrong thing.** A pagination test failed and I first suspected a data race. The real cause was my own locator pattern, which missed products created at runtime because their ids are lowercase. Checking the evidence before changing the test saved me from hiding a bug in a retry.
- **Screenshots depend on fonts.** Visual checks passed locally and failed in CI because the two environments render text with different fonts. The fix was to run the comparison in the same container image that made the baselines.

What the tests found in the application

Six real defects, each documented as a test:

Area	Defect
Admin brands	Search results are cached for an hour and not cleared on delete
Payment	An unsupported payment method is accepted as "Payment was successful"
Admin edit product	The form doesn't pre-fill stock, brand, category or image
Admin add user	Leaving two optional fields blank makes the server answer 500
Orders	An order is created for a cart with no items
Accessibility	A button on the login page has no accessible name

What it looks like now

- 86 tests by default, covering authentication, catalog, cart and checkout, access control, payment rules, the admin area, API response shapes, accessibility and a few visual checks. Three more run on a phone-sized screen.
- The same suite passes on Chromium, Firefox and WebKit in GitHub Actions.
- Every run produces an Allure report, published to GitHub Pages, with each test labelled by area, layer and severity, and each page action shown as a readable step.
- Lint, type checking and formatting are enforced both before commit and in CI.
- For work in regulated areas, an optional mode links every test to a requirement, records a timestamped screenshot and video of every test, and builds a validation pack: one report, a traceability matrix and a checksum for every file. It is separate from the everyday reports and built only on request.

Using it for your project

If your team is starting automation, or has a suite that has become unreliable, this is the shape I'd begin with. Replacing the page objects, API clients and data builders with your application's gets you the same structure: environment profiles, tagged suites, self-cleaning data, reports and a CI pipeline.

I'm available for freelance work in test automation, from framework design to getting an existing suite stable in CI. You can reach me through my GitHub profile.